PulseCheck

API Monitoring Dashboard

1. About PulseCheck

PulseCheck is a lightweight web application that checks whether HTTP/HTTPS endpoints respond, measures response time, and displays HTTP status codes. It helps developers perform a quick, manual health check from one dashboard.

2. System Requirements

2.1 Functional Requirements

- **FR-01:** The system shall allow users to enter up to 10 HTTP/HTTPS URLs for checking.
- **FR-02:** The system shall validate the format of each entered URL.
- **FR-03:** The system shall send HTTP GET requests to the specified endpoints.
- **FR-04:** The system shall measure and display the response time of each request in milliseconds.
- **FR-05:** The system shall display the HTTP status code returned by each endpoint.
- **FR-06:** The system shall classify responses with status codes from 200 to 399 as Online and other HTTP responses as Issue.
- **FR-07:** The system shall display an error message when a request fails or an endpoint cannot be reached.
- **FR-08:** The system shall present the check results in a dashboard.

2.2 Non-Functional Requirements

- **NFR-01:** The system shall apply a timeout of five seconds to each HTTP request.
- **NFR-02:** The system shall provide a simple and user-friendly interface.
- **NFR-03:** The system shall handle request errors without terminating the application.

3. Technologies Used

Technology	Purpose
Python	Implements the checking logic and handles results.
Flask	Runs the web application and handles browser requests.
Requests	Sends HTTP GET requests to the selected endpoints.
HTML	Defines the dashboard page structure.
CSS	Styles the dashboard layout and visual elements.

4. How to Run

Requirements: Python 3.10 or later and an internet connection.

1. Extract the project ZIP file.
2. Open the project folder in Visual Studio Code.
3. Open a terminal in the project folder.
4. Install the required packages:

```
pip install -r requirements.txt
```

Start the application:

```
python app.py
```

Then open http://127.0.0.1:5000 in a web browser and click “Run health check.”

5. System Architecture

The user enters endpoint URLs in the browser. Flask receives the form submission and calls the health-check function. The function uses the Requests library to send an HTTP GET request to each endpoint, measures response time, and records the status or error. Flask then renders the results in the dashboard.

5.1 Components Digram

The Component Diagram illustrates the main components of the PulseCheck system and their interactions. The Flask application handles browser requests and renders the HTML template. The HTTP Checker validates and checks endpoint URLs using the Requests library, which sends HTTP GET requests to external websites and APIs and returns their responses. The HTML template and CSS define the structure and appearance of the dashboard, where the user can view the health-check results. **Fig1**

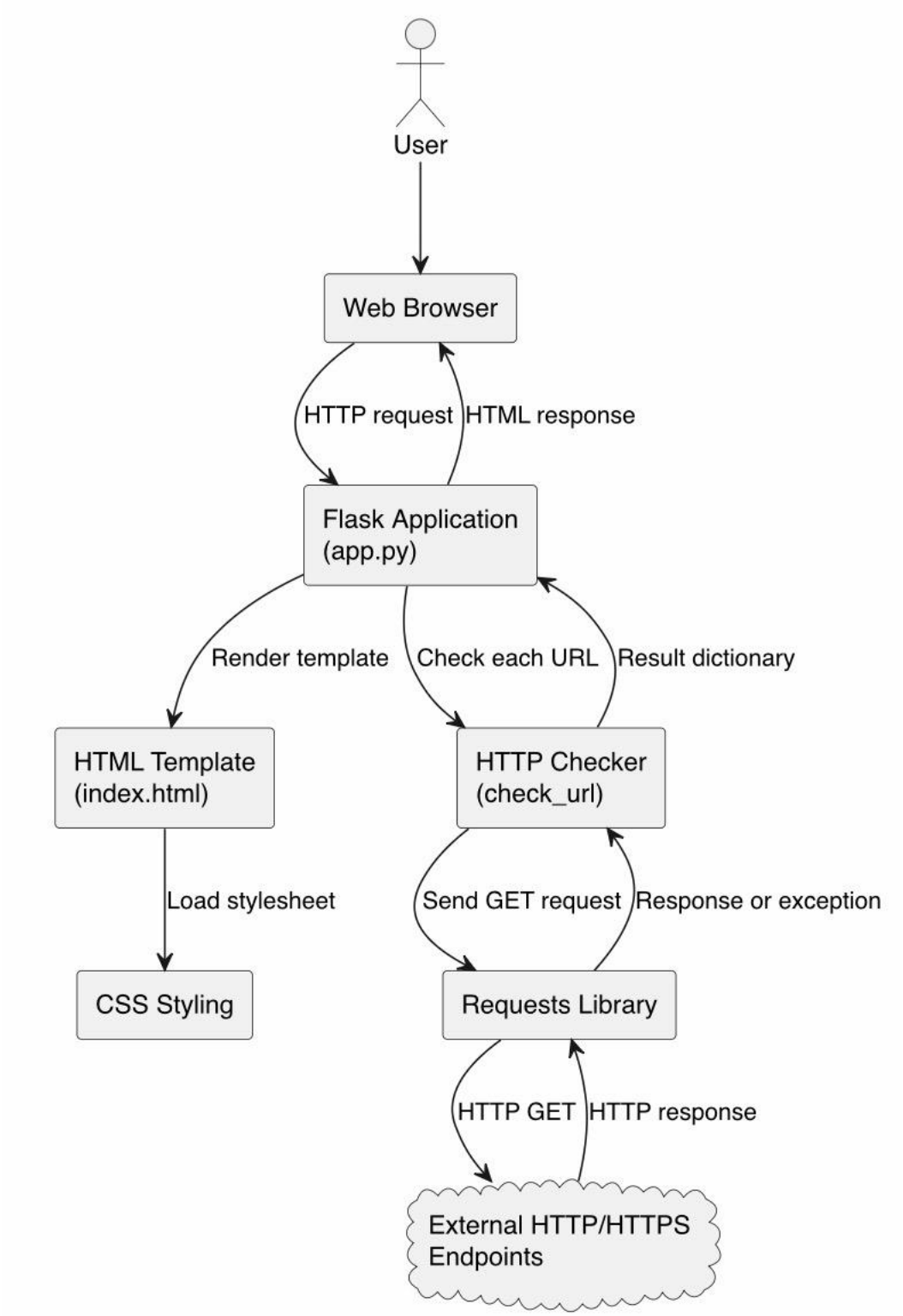


Fig1: Components Digram

5.2 Deployment Digram

The Deployment Diagram illustrates the physical deployment environment of the PulseCheck system. The application runs on an application host using the Python runtime, Flask framework, Health Checker, and Requests library. The user accesses the application through a web browser over HTTP/HTTPS. The Requests library sends outbound HTTP/HTTPS requests to external websites and APIs and receives their responses, allowing the system to display endpoint health-check results to the user. **Fig2**

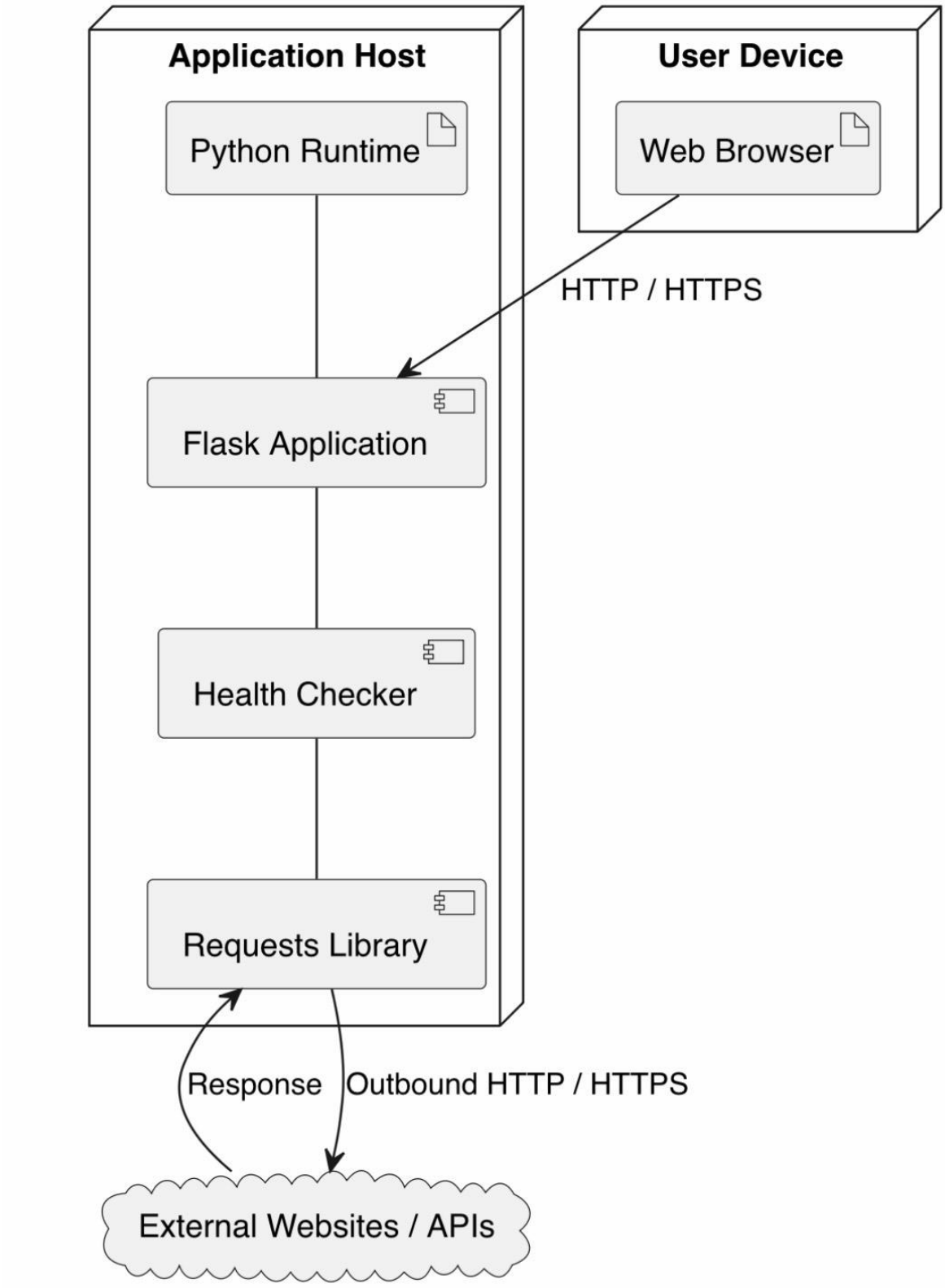


Fig2: Deployment Digram

5.3.1 Use Case Digram

The Use Case Diagram illustrates the main functionalities of the PulseCheck system and how the user interacts with them. The user can enter endpoint URLs, run a health check, and view the health report. The system measures response time and checks HTTP status during the health-check process. The user can also view errors when issues are detected. The diagram provides an overview of the system's main functions and their relationships. **Fig 3**

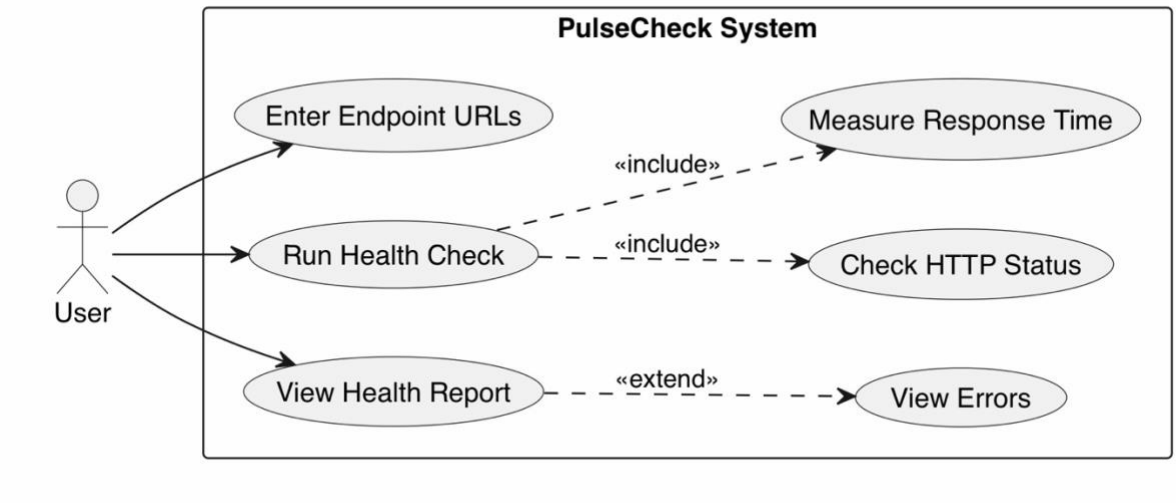


Fig3: Use Case Digram

5.3.2 Use Case Description

Table1: Use Casees Description			
Use Case ID	**Use Case Name**	**Actor**	**Description**
UC-01	Enter Endpoint URLs	User	Allows the user to enter the URLs of endpoints to be checked.
UC-02	Run Health Check	User	Initiates the health check for the entered endpoint URLs.
UC-03	Measure Response Time	System	Measures the time taken for an endpoint to respond.
UC-04	Check HTTP Status	System	Checks the HTTP status code returned by each endpoint.
UC-05	View Health Report	User	Displays the health-check results, including endpoint status and response time.
UC-06	View Errors	User	Displays errors encountered during the health check, when available.

5.4 Activity Digram

The Activity Diagram illustrates the workflow of the PulseCheck system, starting when the user enters endpoint URLs and clicks “Run Health Check.” The system validates each URL and sends an HTTP GET request if the URL is valid. It measures the response time, checks the HTTP status code, and identifies connection errors. Finally, the system classifies each endpoint’s status and displays the results on the dashboard. Invalid URLs are also identified and displayed to the user. **Fig4**

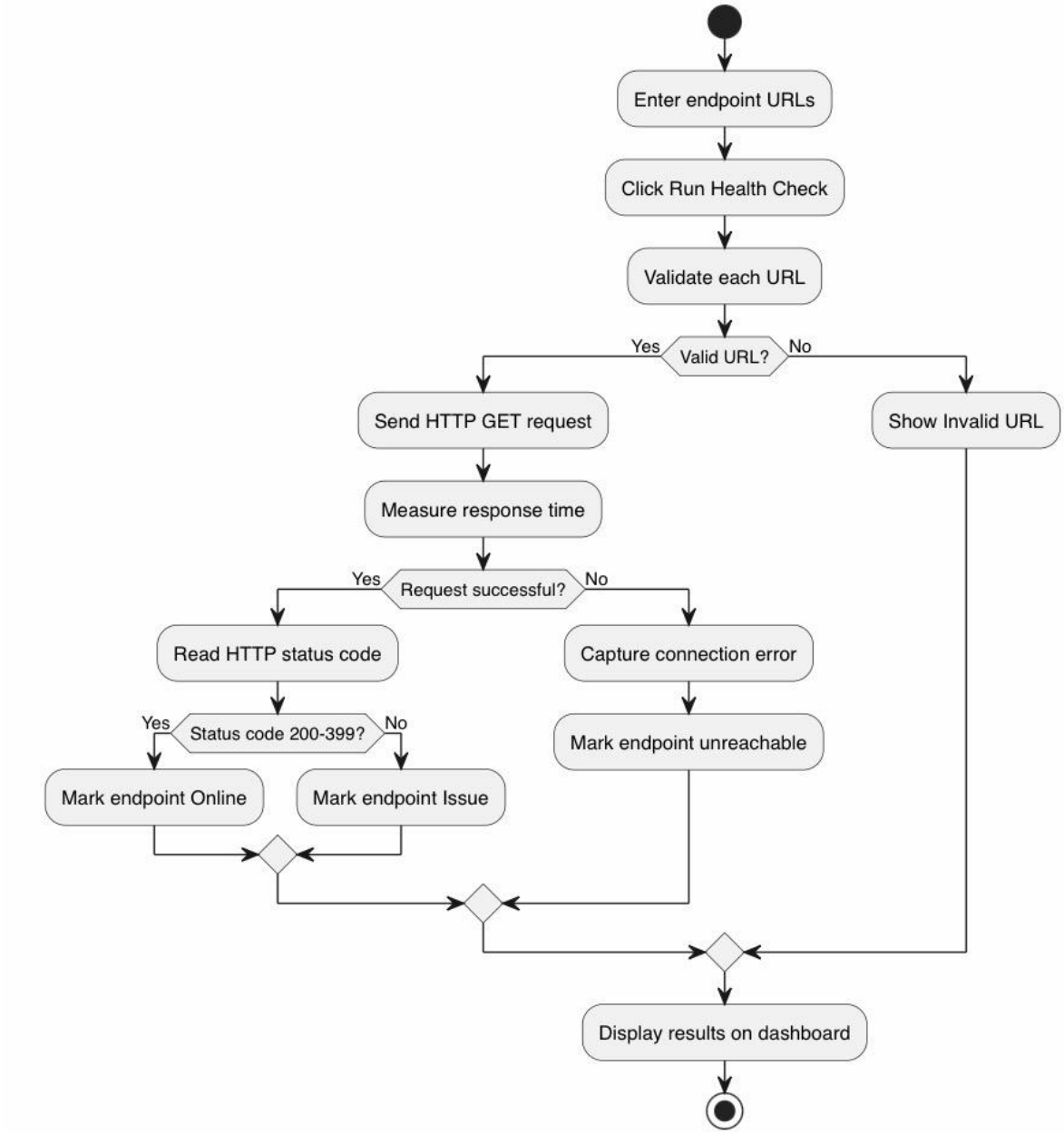


Fig4: Activity Digram

6. Limitations

PulseCheck performs a manual response check; it does not monitor endpoints continuously. An HTTP status check does not verify every feature or the security of a website. Some websites may block automated requests or return responses that

differ from a normal browser visit. Only test endpoints you own or are authorized to check.

7. Possible Future Improvements

- Schedule recurring checks and keep a history of results.
- Add notifications when an endpoint becomes unavailable.
- Create automated tests for API response bodies and expected data.